

Distributed Systems

CS 111

Fall 2024

Operating System Principles

Peter Reiher

Outline

- Introduction
- Distributed system paradigms
- Distributed synchronization
- Distributed system security
- Accessing remote data

Introduction

- Why do we care about distributed systems?
 - Because that's how most modern computing is done
- Why is this an OS topic?
 - Because it's definitely a systems issue
 - And even the OS on a single computer needs to worry about distributed issues
- If you don't know a bit about distributed systems, you're not a modern computer scientist

Why Distributed Systems?

- Better scalability and performance
 - Apps require more resources than one computer has
 - Can we grow system capacity/bandwidth to meet demand?
- Improved reliability and availability
 - 24x7 service despite disk/computer/software failures
- Ease of use, with reduced operating expenses
 - Centralized management of all services and systems
 - Buy (better) services rather than computer equipment
- Enabling new collaboration and business models
 - Collaborations that span system (or national) boundaries
 - A global free market for a wide range of new services

A Few Little Problems

- Different machines don't share memory
 - Or any peripheral devices
 - So one machine can't easily know the state of another

Might this cause synchronization problems?
- The only way to interact remotely is to use a network
 - Usually asynchronous, slow, and error prone
 - Usually not controlled by any single machine

So how can we know what's going on remotely?
- Failures of one machine aren't visible to other machines
 - How can our computation be reliable if pieces fail?

Transparency

- Ideally, a distributed system would be just like a single machine system
- But better
 - More resources
 - More reliable
 - Faster
- *Transparent* distributed systems look as much like single machine systems as possible

Deutsch's “Seven Fallacies of Network Computing”

1. The network is reliable
2. There is no latency (instant response time)
3. The available bandwidth is infinite
4. The network is secure
5. The topology of the network does not change
6. There is one administrator for the whole network
7. The cost of transporting additional data is zero

Here's an eight:
all locations on
the network are
equivalent.

Bottom Line: true transparency is not achievable

Distributed System Paradigms

- Parallel processing. ○ ○
 - Relying on tightly coupled special hardware
- Single system images^{○○}
 - Make all the nodes look like one big computer
 - Somewhere between hard and impossible
- Loosely coupled systems
 - Work with difficulties as best as you can
 - Typical modern approach to distributed systems
- Cloud computing
 - A recent variant

Not widely used, we won't discuss them.

So these are also not popular, and we won't discuss them.

Loosely Coupled Systems

- Characterization:
 - A parallel group of independent computers
 - Connected by a high speed LAN
 - Serving similar but independent requests
 - Minimal coordination and cooperation required
- Motivation:
 - Scalability and price performance
 - Availability – if protocol permits stateless servers
 - Ease of management, reconfigurable capacity
- Examples:
 - Web servers, app servers

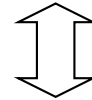
Horizontal Scalability

- Each node largely independent
- So you can add capacity just by adding a node “on the side”
- Scalability can be limited by network, instead of hardware or algorithms
 - Or, perhaps, by a load balancer
- Reliability is high
 - Failure of one of N nodes just reduces capacity

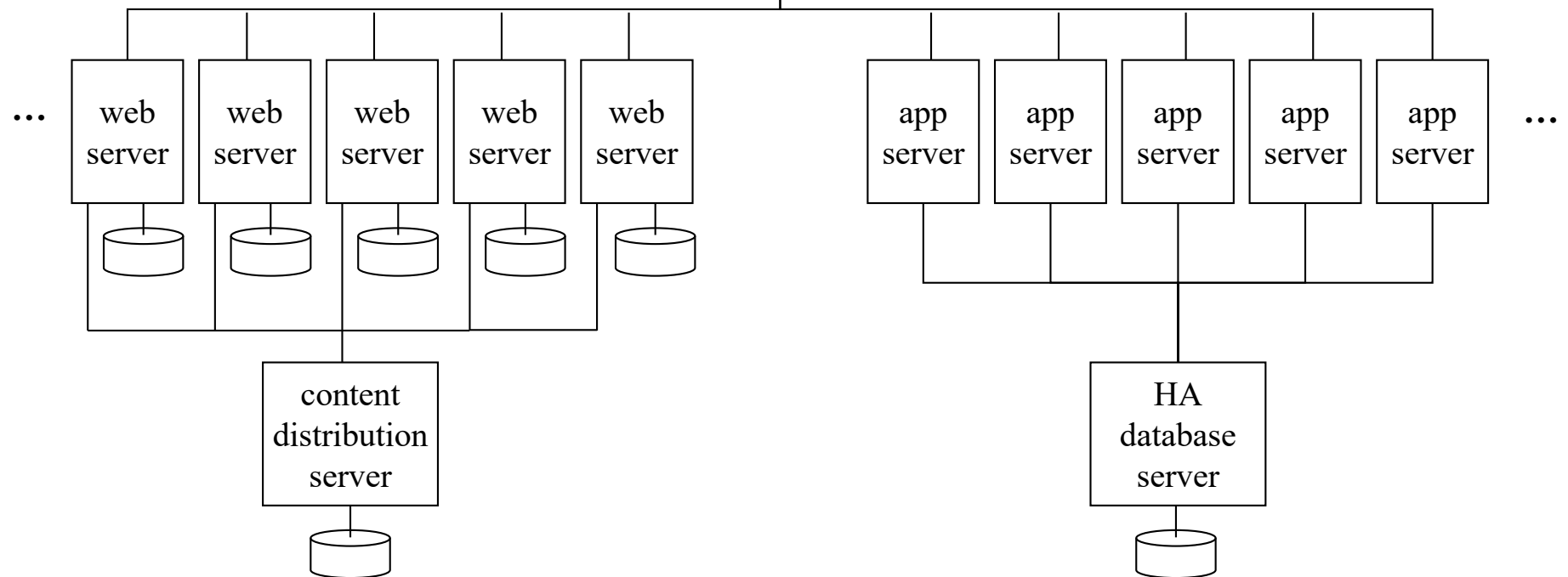
Horizontal Scalability Architecture

If I need more
web server
capacity,

WAN to clients



load balancing switch
with fail-over



Elements of Loosely Coupled Architecture

- Farm of independent servers
 - Servers run same software, serve different requests
 - May share a common back-end database
- Front-end switch
 - Distributes incoming requests among available servers
 - Can do both load balancing and fail-over
- Service protocol
 - Stateless servers and *idempotent* operations
 - Successive requests may be sent to different servers

Same result if you do it once,
twice, three times, . . . , n times

Horizontally Scaled Performance

- Individual servers are very inexpensive
 - Blade servers may be only \$100-\$200 each
- Scalability is excellent
 - 100 servers deliver approximately 100x performance
- Service availability is excellent
 - Front-end automatically bypasses failed servers
 - Stateless servers and client retries fail-over easily
- The challenge is managing thousands of servers
 - Automated installation, global configuration services
 - Self monitoring, self-healing systems
 - Scaling limited by management, not HW or algorithms

Cloud Computing

- The most recent twist on distributed computing
- We discussed this a couple lectures back
- What runs in a cloud?
- In principle, anything
 - But general distributed computing is hard
- So much of the work is run using special tools
- These tools support particular kinds of parallel/distributed processing
 - Using a method like map-reduce or horizontal scaling
- So the user need not be a distributed systems expert

MapReduce

- Perhaps the most common cloud computing software tool/technique
- A method of dividing large problems into compartmentalized pieces
- Each of which can be performed on a separate node
- With an eventual combined set of results

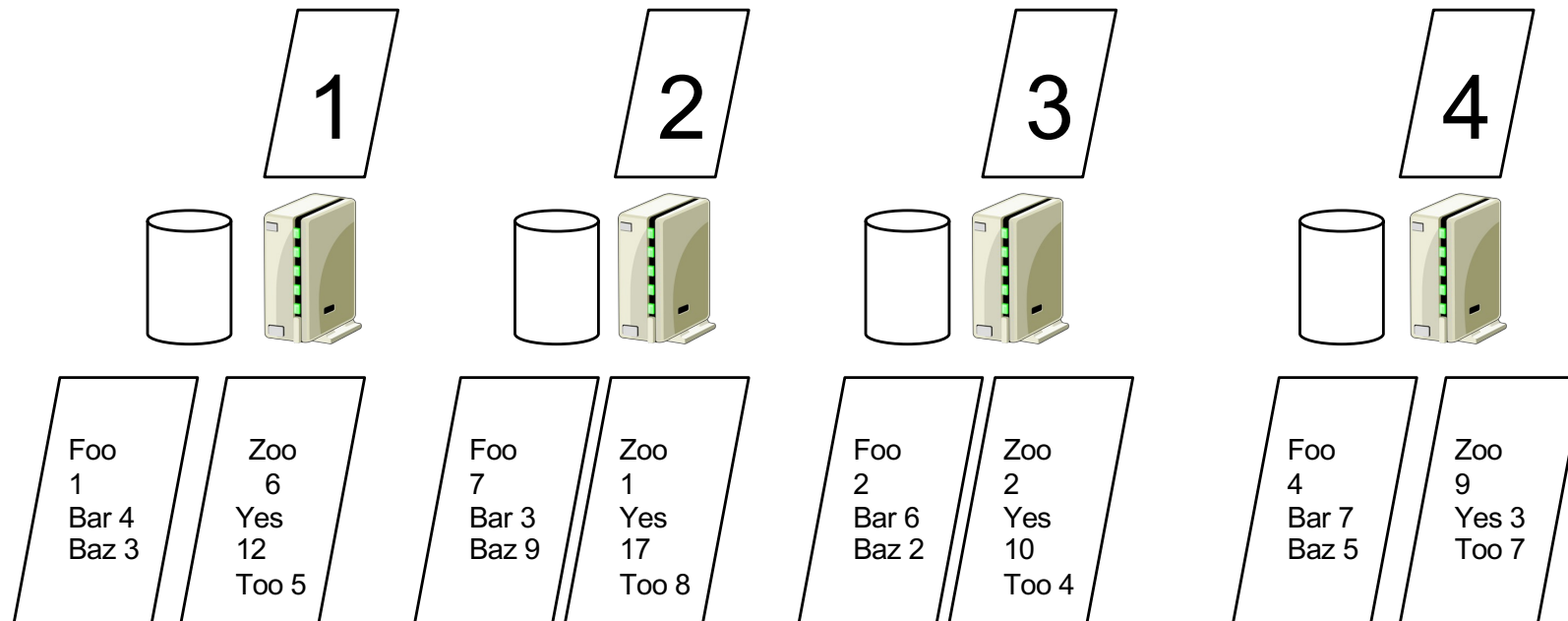
The Idea Behind MapReduce

- There is a single function you want to perform on a lot of data
 - Such as searching it for a particular string
- Divide the data into disjoint pieces
- Perform the function on each piece on a separate node (*map*)
- Combine the results to obtain output (*reduce*)

An Example

- We have 64 megabytes of text data
- Count how many times each word occurs in the text
- Divide it into 4 chunks of 16 Mbytes
- Assign each chunk to one processor
- Perform the map function of “count words” on each

The Example Continued

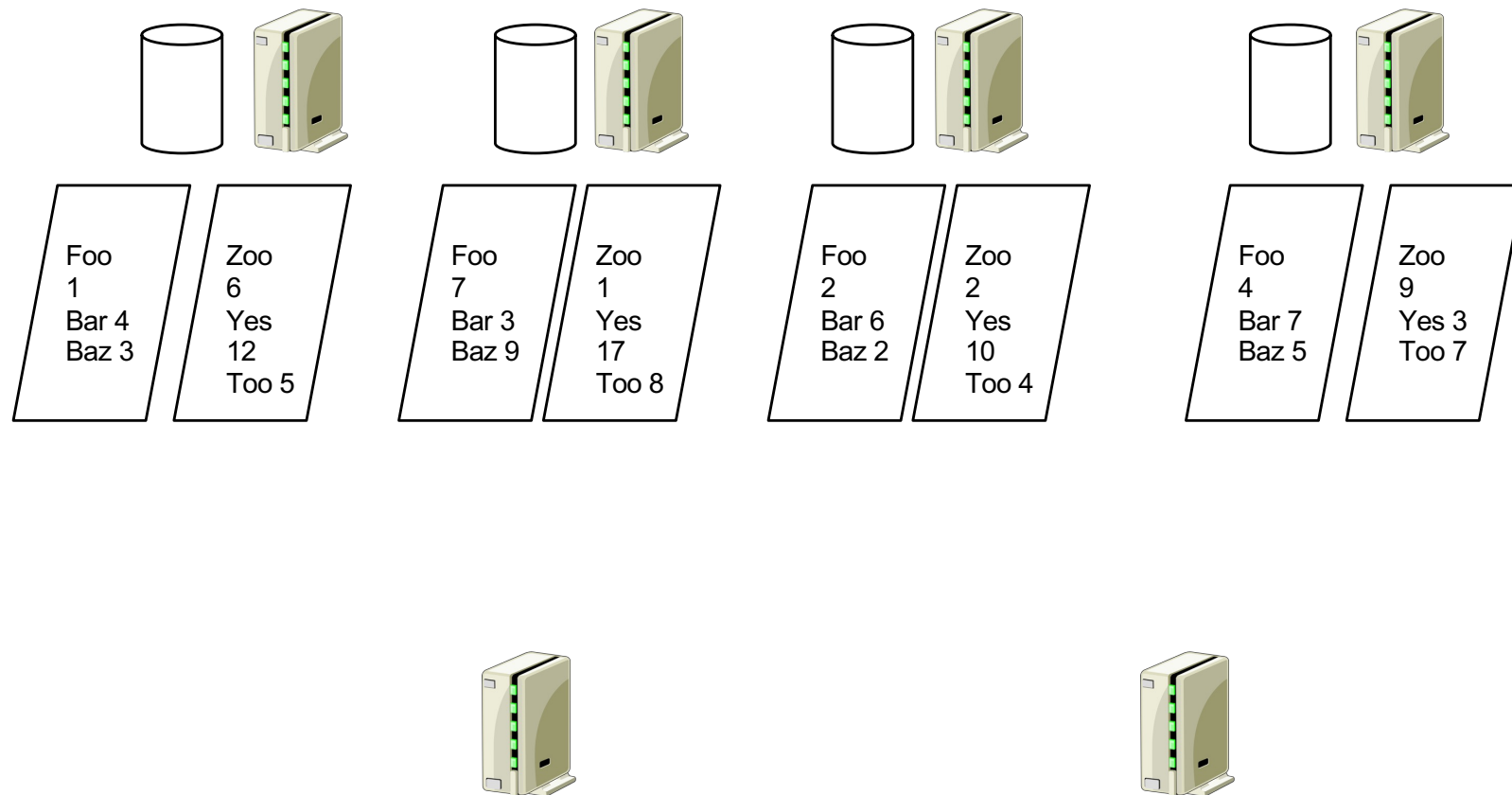


That's the map stage

On To Reduce

- We might have two more nodes assigned to doing the reduce operation
- They will each receive a share of data from a map node
- The reduce node performs a reduce operation to “combine” the shares
- Outputting its own result

Continuing the Example



The Reduce Nodes Do Their Job

Write out the results to files
And MapReduce is done!



Foo
14
Bar 20
Baz
19



Zoo
16
Yes
42
Too
24

But I Wanted A Combined List

- No problem
- Run another (slightly different) MapReduce on the outputs
- Have one reduce node that combines everything

Synchronization in MapReduce

- Each map node produces an output file for each reduce node
- It is produced atomically
- The reduce node can't work on this data until the whole file is written
- Forcing a synchronization point between the map and reduce phases

Cloud Computing and Horizontal Scaling

- An excellent match
- Rent some cloud nodes to be your web servers
- If load gets heavy, ask the cloud for another web server node
- As load lightens, release unneeded nodes
- No need to buy new machines
- No need to administer your own machines

Remote Procedure Calls

- RPC, for short
- One way of building a distributed program
- Procedure calls are a fundamental paradigm
 - Primary unit of computation in most languages
 - Unit of information hiding in most methodologies
 - Primary level of interface specification
- A natural boundary between client and server
 - Turn procedure calls into message send/receives
- A few limitations
 - No implicit parameters/returns (e.g., global variables)
 - No call-by-reference parameters
 - Much slower than procedure calls (TANSTAAFL)

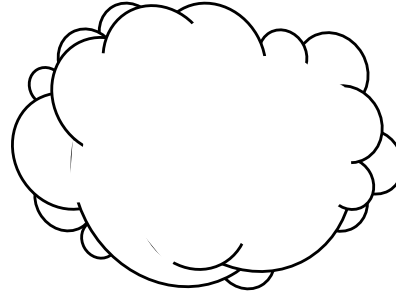
Remote Procedure Call Concepts

- Interface Specification
 - Methods, parameter types, return types
- eXternal Data Representation (XDR)
 - Machine independent data-type representations
 - May have optimizations for similar client/server
- Client stub
 - Client-side proxy for a method in the API
- Server stub (or skeleton)
 - Server-side recipient for API invocations

Key Features of RPC

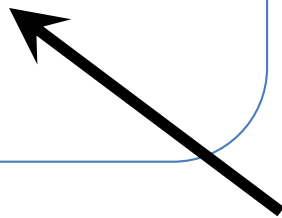
- Client application links against local procedures
 - Calls local procedures, gets results
- All RPC implementation inside those procedures
- Client application does not know about RPC
 - Does not know about formats of messages
 - Does not worry about sends, timeouts, resends
 - Does not know about external data representation
- All of this is generated automatically by RPC tools
- The key to the tools is the interface specification

RPC At Work, Step 1



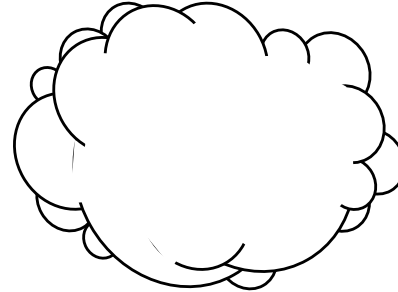
Process_list <return>

```
. . .  
list[0] = 10;  
list[1] = 20;  
list[2] = 17;  
max = list_max(list);
```



`list_max()` is a remote procedure call!

RPC At Work, Step 2



Process_list <return>

```
. . .  
list[0] = 10;  
list[1] = 20;  
list[2] = 17;  
max = list_max(list);
```



Format RPC message

```
RPC message: list_max(),  
parameter list
```

Send the message



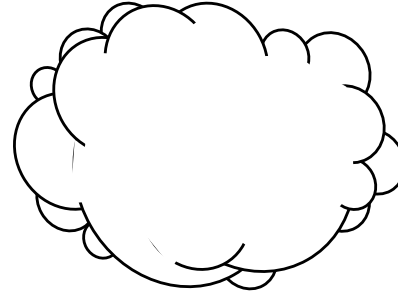
Extract RPC info

```
list_max()
```

```
list
```

Call local procedure

RPC At Work, Step 3



```
. . .  
list[0] = 10;  
list[1] = 20;  
list[2] = 17;  
max = list_max(list);  
If (max > 10) {
```

max

20



Extract the return value

Resume the local program



```
local_max =  
list_max(list);
```

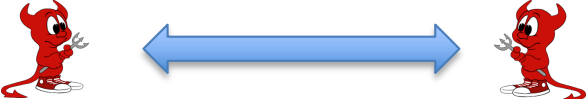
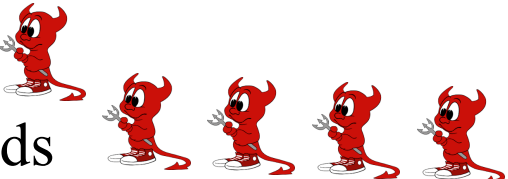
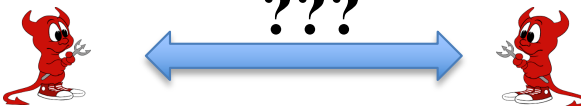
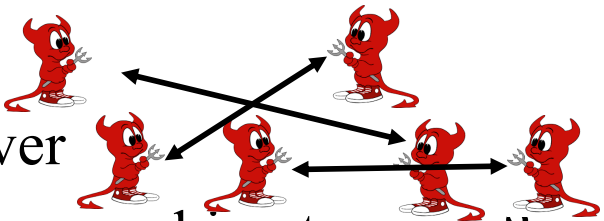
local_max 20

Format RPC response

RPC response: list_max(),
return value 20

Send the message

RPC Is Not a Complete Solution

- Requires client/server binding model 
 - Expects to be given a live connection
- Threading model implementation for RPC servers 
 - A single thread services requests one at a time
 - So use numerous one-per-request worker threads
- Limited failure handling 
 - Client must arrange for timeout and recovery
- Limited consistency support 
 - Only between calling client and called server
 - What if there are multiple clients and servers working together?
- Higher level abstractions improve RPC
 - e.g. Microsoft DCOM, Java RMI, DRb, Pyro

Security for Distributed Systems

- Security is hard in single machines
- It's even harder in distributed systems
- Why?

Why Is Distributed Security Harder?

- Your OS cannot guarantee privacy and integrity
 - Network activities happen outside of the OS
 - Should you trust where they happen?
- Authentication is harder
 - All possible agents may not be in local password file
- The wire connecting the user to the system is insecure
 - Eavesdropping, replays, man-in-the-middle attacks
- Even with honest partners, hard to coordinate distributed security
- The Internet is an open network for all
 - Many sites on the Internet try to serve all comers
 - Core Internet makes no judgments on what's acceptable
 - Even supposedly private systems may be on Internet

Goals of Network Security

- Secure conversations
 - Privacy: only you and your partner know what is said
 - Integrity: nobody can tamper with your messages
- Positive identification of both parties
 - Authentication of the identity of message sender
 - Assurance that a message is not a replay or forgery
 - Non-repudiation: he cannot claim “I didn't say that”
- Availability
 - The network and other nodes must be reachable when they need to be

Elements of Network Security

- Cryptography
 - Symmetric cryptography for protecting bulk transport of data
 - Public key cryptography primarily for authentication
 - *Cryptographic hashes* to detect message alterations
- Digital signatures and public key certificates
 - Powerful tools to authenticate a message's sender
- Filtering technologies
 - Firewalls and the like
 - To keep bad stuff from reaching our machines

Tamper Detection: Cryptographic Hashes

- Check-sums often used to detect data corruption
 - Add up all bytes in a block, send sum along with data
 - Recipient adds up all the received bytes
 - If check-sums agree, the data is probably OK
 - Check-sum (parity, CRC, ECC) algorithms are weak
- Cryptographic hashes are very strong check-sums
 - Unique –two messages vanishingly unlikely to produce same hash
 - Particularly hard to find two messages with the same hash
 - One way – cannot infer original input from output
 - Well distributed – any change to input changes output

Easy to create a different message with the
same checksum

Using Cryptographic Hashes

- Start with a message you want to protect
- Compute a cryptographic hash for that message
 - E.g., using the Secure Hash Algorithm 3 (SHA-3)
- Transmit the hash securely
- Recipient does same computation on received text
 - If both hash results agree, the message is intact
 - If not, the message has been corrupted/compromised

Secure Hash Transport

- Why must the hash be transmitted securely?
 - Cryptographic hashes aren't keyed, so anyone can produce them (including a bad guy)
- How to transmit hash securely?
 - Encrypt it
 - Unless secrecy required, cheaper than encrypting entire message
 - If you have a secure channel, could transmit it that way
 - But if you have secure channel, why not use it for everything?

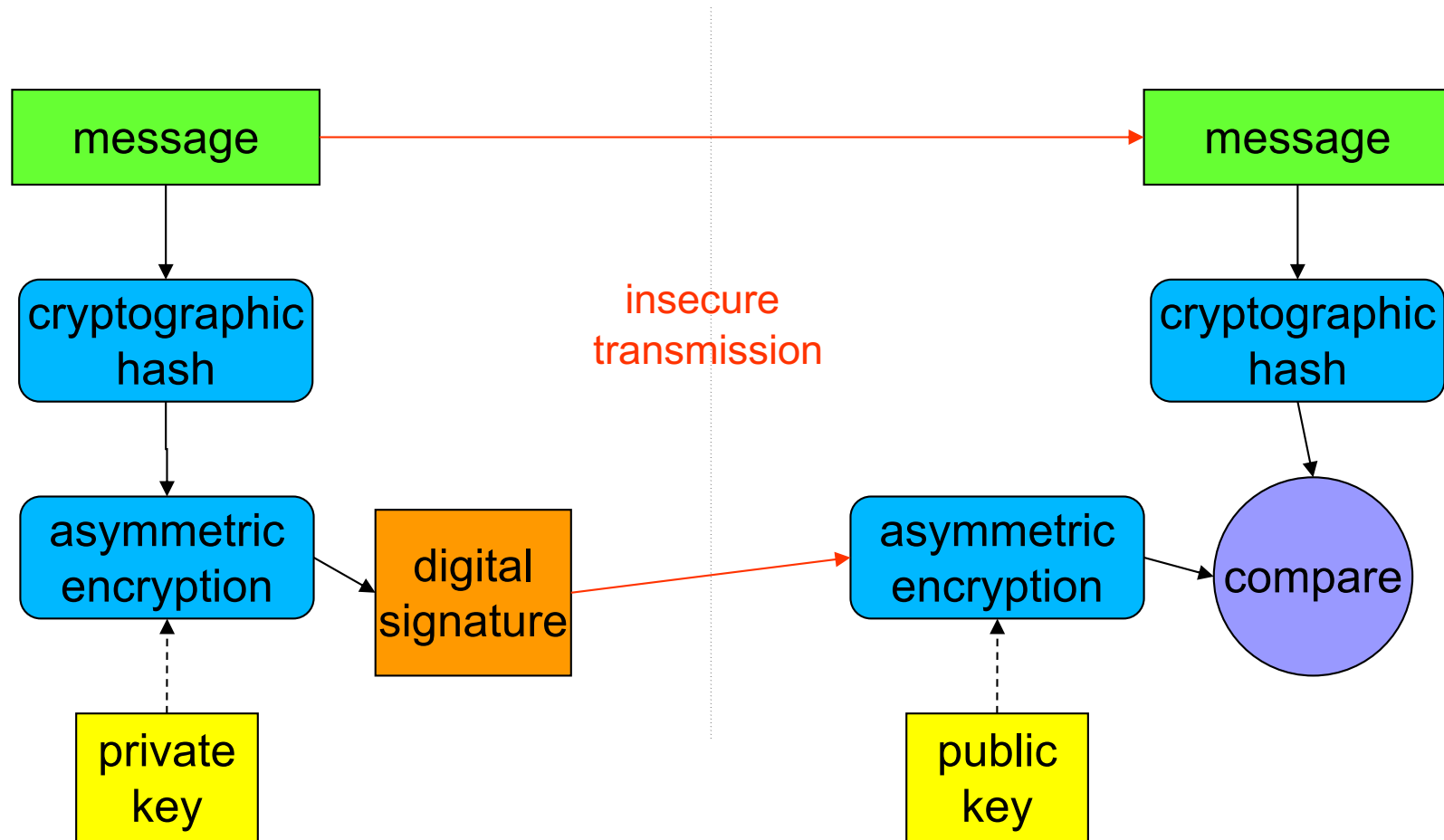
Putting It Together: Secure Socket Layer (SSL)

- A general solution for securing network communication
- Built on top of existing socket IPC
- Establishes secure link between two parties
 - Privacy – nobody can snoop on conversation
 - Integrity – nobody can generate fake messages
- Certificate-based authentication of server
 - Typically, but not necessarily
 - Client knows what server he is talking to
- Optional certificate-based authentication of client
 - If server requires authentication and non-repudiation
- PK used to distribute a symmetric session key
 - New key for each new socket
- Rest of data transport switches to symmetric crypto
 - Giving safety of public key and efficiency of symmetric

Digital Signatures

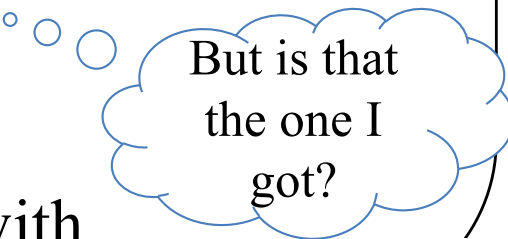
- Encrypting a message with private key signs it
 - Only you could have encrypted it, it must be from you
 - It has not been tampered with after you wrote it
- Encrypting everything with your private key is a bad idea
 - Asymmetric encryption is extremely slow
- If you only care about integrity, you don't need to encrypt it all
 - Compute a cryptographic hash of your message
 - Encrypt the cryptographic hash with your private key
 - Faster than encrypting whole message

Digital Signatures



Signed Load Modules

- How do we know we can trust a program?
 - Is it really the new update to Windows, or actually evil code that will screw me?
 - Digital signatures can answer this question
- Designate a certification authority
 - Perhaps the OS manufacturer (Microsoft, Apple, ...)
- They verify the reliability of the software
 - By code review, by testing, etc.
 - They sign a certified module with their private key
- We can verify signature with their public key
 - Proves the module was certified by them
 - Proves the module has not been tampered with



But is that
the one I
got?

An Important Public Key Issue

- If I have a public key
 - I can authenticate received messages
 - I know they were sent by the owner of the private key
- But how can I be sure who owns the private key?
 - How do I know that this is really my bank's public key?
 - Could some swindler have sent me his public key instead?
- I can get Microsoft's public key when I first buy their OS
 - So I can verify their load modules and updates
 - But how to handle the more general case?
- I would like a certificate of authenticity
 - Guaranteeing who the real owner of a public key is

What Is a PK Certificate?

- Essentially a data structure
- Containing an identity and a matching public key
 - And perhaps other information
- Also containing a digital signature of those items
- Signature usually signed by someone I trust
 - And whose public key I already have

Using Public Key Certificates

- If I know public key of the authority who signed it
 - I can validate that the signature is correct
 - And that the certificate has not been tampered with
- If I trust the authority who signed the certificate
 - I can trust they authenticated the certificate owner
 - E.g., we trust drivers licenses and passports
- But first I must know and trust signing authority
 - Which really means I know and trust their public key

A Chicken and Egg Problem

- I can learn the public key of a new partner using his certificate
- But to use his certificate, I need the public key of whoever signed it
- So how do I get that public key?
- Ultimately, *out of band*
 - Which means through some other means
- Commonly by having the key in a trusted program, like a web browser
- Or hand delivered

Conclusion

- Distributed systems offer us much greater power than one machine can provide
- They do so at costs of complexity and security risk
- We handle the complexity by using distributed systems in a few carefully defined ways
- We handle the security risk by proper use of cryptography and other tools